SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
COMPUTER SCIENCE DEPARTMENT

SPRING 2025

Design and Analysis of Algorithms.
Assignment 1

Semester: 6th — Batch: 2022F
Name: Muhammad Faisal — Section: A
Roll #: 2022F-BCS-152

Q1) Define the process involved in the analysis of bubble sort algorithm through derivation. Code algorithm in C++ or C# or Python

Bubble Sort is a comparison-based and in-place sorting algorithm.

- It repeatedly steps through the list.
- Compares adjacent elements.
- Swaps them if they are in the wrong order.
- After each pass, the largest unsorted element "bubbles" up to its correct position at the end.

Step-by-Step Working with Example

Let's take an example:
Input Array: [5, 1, 4, 2, 8]

Pass 1:

- Compare 5 & 1 → Swap → [**1**, **5**, 4, 2, 8]
- Compare 5 & 4 → Swap → [1, **4**, **5**, 2, 8]
- Compare 5 & 2 → Swap → [1, 4, **2**, **5**, 8]
- Compare 5 & 8 → No swap

Result after Pass 1: [1, 4, 2, 5, 8]
Biggest element (8) is in correct place.

Pass 2:

- Compare 1 & 4 → No swap
- Compare 4 & 2 → Swap → [1, **2**, **4**, 5, 8]
- Compare 4 & 5 → No swap

Result after Pass 2: [1, 2, 4, 5, 8]

Pass 3:

- Compare 1 & 2 → No swap
- Compare 2 & 4 → No swap

No swaps occurred → Array is sorted → Early exit.

Time Complexity Derivation

```
int BubbleSort(int arr[]) {
  int n = arr.length;        // 1 step
  int temp;                  // 1 step

  for (i = 0; i < n-1; i++) {     // Outer loop runs N-1 times → contributes 2N
    for (j = 0; j < n-1; j++) {  // Inner loop: each i runs (N-1) times → x = (n-1)
      if (arr[j] > arr[j+1]) {    // 3x
        temp = arr[j];           // 2x
        arr[j] = arr[j+1];       // 2x
        arr[j+1] = temp;         // 2x
      }
    }
  }
```

```
  return arr;              // 1 step
}
```

Total Steps:

$T(N) = 1 + 1 + 2N + 2x + 2 + 3x + 2x + 2x + 2x + 1$

$= 11x + 2N + 6$

Where:

$x = 1 + 2 + 3 + ... + (n-1)$

$= (n-1)(n)/2$ [Arithmetic progression]

Plug into T(N):

$T(N) = 11 * (n(n-1)/2) + 2N + 6$

$= (11/2)N^2 - (11/2)N + 2N + 6$

$= O(N^2)$ → Ignoring constants and lower-order terms

Types of Analysis

Best Case:

- Already sorted array
- Comparisons = n-1
- No swaps
- **Time Complexity: O(n)** (with optimization using swapped flag)

Worst Case:

- Reverse sorted array
- Maximum comparisons and maximum swaps
- **Time Complexity: $O(n^2)$**

Average Case:

- Random order
- On average, performs n(n-1)/4 swaps
- **Time Complexity: $O(n^2)$**

C++ Code

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
  bool swapped;

  // Outer loop for passes
  for (int i = 0; i < n - 1; i++) {
    swapped = false;

    // Inner loop for comparisons
    for (int j = 0; j < n - i - 1; j++) {

      // Compare adjacent elements
      if (arr[j] > arr[j + 1]) {

        // Swap if they are in wrong order
        swap(arr[j], arr[j + 1]);
        swapped = true;
      }
    }

    // If no two elements were swapped, stop early
```

```
        if (!swapped)
            break;
    }
}

// Function to print the array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver function
int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);

    cout << "Original array: ";
    printArray(arr, n);

    bubbleSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    return 0;
}
```

```
Original array: 64 34 25 12 22 11 90
Sorted array: 11 12 22 25 34 64 90
```

Q2) Define the process involved in the analysis of Insertion Sort Algorithm through derivation. Code algorithm in C++ or C# or Python

Insertion Sort builds the sorted array one element at a time by:

- Picking the next element (called key),
- Comparing it with all previous elements in the sorted part of the array,
- And inserting it into the correct position.

Step-by-Step Working with Example

Input: [5, 2, 4, 6, 1, 3]

We assume the **first element is already sorted**.

Pass 1 (key = 2)

- Compare with 5 → 5 > 2 → shift 5
- Insert 2 at correct position

  [2, 5, 4, 6, 1, 3]

Pass 2 (key = 4)

- Compare with 5 → 5 > 4 → shift 5
- Compare with 2 → 2 < 4 → insert 4

  [2, 4, 5, 6, 1, 3]

Pass 3 (key = 6)

- No shifting needed

  [2, 4, 5, 6, 1, 3]

Pass 4 (key = 1)

- Shift 6, 5, 4, 2
- Insert 1 at index 0

  [1, 2, 4, 5, 6, 3]

Pass 5 (key = 3)

- Shift 6, 5, 4
- Insert 3 between 2 and 4
  [1, 2, 3, 4, 5, 6]

Step-by-Step Time Complexity Derivation

```
int insertion_sort(int a[], int n) {
   int i;       // 1
   int j;       // 1
   int temp;    // 1

   for(i = 0; i < n; i++) {          // 1 + N + (N - 1) = 2N
      temp = a[i];                   // 2(N - 1)
      j = i - 1;                     // 2(N - 1)

      while(j >= 0 && a[j] > temp) {  // 3x  (x depends on data)
         a[j + 1] = a[j];             // 2x
         j--;                         // x
      }

      a[j + 1] = temp;                // 2(N - 1)
   }
}
```

Total T(N):

$T(N) = 1 + 1 + 1$
$+ 2N$
$+ 2(N - 1) + 2(N - 1)$
$+ 3x + 2x + x$
$+ 2(N - 1)$

$T(N) = 3 + 2N + 4(N - 1) + 6x + 2(N - 1)$
$= 3 + 2N + 4N - 4 + 6x + 2N - 2$
$= 6x + 8N - 3$

Worst Case Analysis

In the worst case (array in reverse order), the inner loop runs maximum times:
$x = 1 + 2 + 3 + ... + (N - 1) = N(N - 1)/2$
Substitute x:
$T(N) = 6x + 8N - 3$
$= 6(N(N - 1)/2) + 8N - 3$
$= 3N^2 - 3N + 8N - 3$
$= 3N^2 + 5N - 3$

Time Complexity in Big-O Notation

$T(N) = O(N^2)$

Best-Case:

- Already sorted
- Only one comparison per iteration
- No shifts
- **Time Complexity: O(n)**

Average Case:

- Random order
- $\sim n^2/4$ comparisons and shifts
- **Time Complexity: $O(n^2)$**

C++ Code

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
   for (int i = 1; i < n; i++) {
      int key = arr[i];  // Current element to be inserted
      int j = i - 1;

      // Shift elements greater than key to the right
      while (j >= 0 && arr[j] > key) {
         arr[j + 1] = arr[j];
         j--;
      }

      // Insert the key at correct position
      arr[j + 1] = key;
   }
}

// Function to print array
void printArray(int arr[], int n) {
   for (int i = 0; i < n; i++)
      cout << arr[i] << " ";
   cout << endl;
}

// Main driver
int main() {
   int arr[] = {5, 2, 4, 6, 1, 3};
   int n = sizeof(arr) / sizeof(arr[0]);

   cout << "Original array: ";
   printArray(arr, n);

   insertionSort(arr, n);

   cout << "Sorted array: ";
   printArray(arr, n);

   return 0;
}
```

```
Original array: 5 2 4 6 1 3
Sorted array: 1 2 3 4 5 6
```

Q3) Define the process involved in the analysis of Selection Sort Algorithm through derivation. Code algorithm in C++ or C# or Python

Selection Sort is a simple comparison-based sorting algorithm.
It repeatedly finds the minimum element from the unsorted part and places it at the beginning.

Step-by-Step Working with Example

Input: [64, 25, 12, 22, 11]

Pass 1:

- Find min from index 0 to 4 → 11

- Swap with index 0
  [11, 25, 12, 22, 64]

Pass 2:

- Find min from index 1 to 4 → 12
- Swap with index 1
  [11, 12, 25, 22, 64]

Pass 3:

- Find min from index 2 to 4 → 22
- Swap with index 2
  [11, 12, 22, 25, 64]

Pass 4:

- Find min from index 3 to 4 → 25 (already min)
- No swap
  [11, 12, 22, 25, 64]

Step-by-Step Derivation of Time Complexity

```
int selectionSort(int arr[], int n) {
   int minindex;     // 1
   int temp;         // 1

   for(int i = 0; i < n - 1; i++) {          // 1 + N-1 + 1 + N-1 = 2N
      minindex = i;                          // N - 1

      for(int j = i + 1; j < n; j++) {       // N - 1 times
         if(arr[j] < arr[minindex]) {        // x times
            minindex = j;                    // x times
         }
      }

      temp = arr[i];                         // 2(N - 1)
      arr[i] = arr[minindex];                // 2(N - 1)
      arr[minindex] = temp;                  // 2(N - 1)
   }

   return 0;                                 // 1
}
```

Total Time T(N):

$T(N) = 1 + 1 + 2N + (N - 1) + (N - 1) + x + 1 + x + 3x + 6(N - 1) + 1$
$= 6x + 10N - 4$

Where:

$x = 1 + 2 + 3 + ... + (N - 1) = N(N - 1)/2$ [Arithmetic Progression]

Substitute:

$T(N) = 6(N(N - 1)/2) + 10N - 4$
$= 3N^2 - 3N + 10N - 4$
$= 3N^2 + 7N - 4$

Time Complexity in Big-O Notation

$T(N) = O(N^2)$

C++ Code

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;

        // Find index of minimum element in unsorted part
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx])
                minIdx = j;
        }

        // Swap minimum element with the first unsorted element
        swap(arr[i], arr[minIdx]);
    }
}

// Function to print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver function
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: ";
    printArray(arr, n);

    selectionSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    return 0;
}
```

```
Original array: 64 25 12 22 11
Sorted array: 11 12 22 25 64
```

Q4) Define the process involved in the analysis of Shell Sort Algorithm through derivation. Code algorithm in C++ or C# or Python

Shell Sort is an optimization of Insertion Sort that allows the exchange of far-apart elements. It sorts elements at a specific gap and reduces the gap step-by-step until it becomes 1 (like regular insertion sort).

Derivation of Time Complexity

```
void shellSort(int arr[], int n) {
    int gap, i, j, temp;              // 4

    for (gap = n/2; gap > 0; gap /= 2) { // ~ log₂(n) passes
        for (i = gap; i < n; i++) {     // Inner loop runs N times (in total across all passes)
            temp = arr[i];             // Assignment
            j = i;

            while (j >= gap && arr[j - gap] > temp) {  // Depends on gap sequence
                arr[j] = arr[j - gap];
                j -= gap;
```

```
        }

        arr[j] = temp;
      }
    }
}
```

Variables:

- Let N be the number of elements
- gaps: the gap sequence (commonly N/2, N/4, ..., 1)
- Let's assume **$\log_2 N$** passes due to halving gap
- Inner while loop can go deep depending on the distribution

Worst Case Time Complexity

Worst-case depends on the gap sequence:

- Using Shell's original sequence: **$O(N^2)$**
- Using Hibbard's or Sedgewick's sequence: **O(N^3/2)** or better

For analysis with Shell's original gap sequence (N/2, N/4, ..., 1):

Operation	Count (Approx)
Outer loop ($\log_2 N$ gaps)	$\log_2 N$
Inner loop (for each gap)	Up to N comparisons
Inner while loop (depends on input)	In worst case → O(N) per pass

So:

$T(N) \approx O(\log N)$ passes $\times O(N^2)$ in worst pass

$T(N) = O(N^2)$ in worst case

Best Case Time Complexity

When the list is already sorted:

- Minimal comparisons
- No shifting

$T(N) = O(N \log N)$

C++ Code

```
#include <iostream>
using namespace std;

void shellSort(int arr[], int n) {
  // Start with a large gap, then reduce the gap
  for (int gap = n / 2; gap > 0; gap /= 2) {
    // Do a gapped insertion sort
    for (int i = gap; i < n; i++) {
      int temp = arr[i];
      int j;

      // Shift earlier gap-sorted elements up until correct location is found
      for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
        arr[j] = arr[j - gap];

      arr[j] = temp;
    }
  }
}

// Function to print the array
void printArray(int arr[], int n) {
  for (int i = 0; i < n; i++)
```

```
    cout << arr[i] << " ";
  cout << endl;
}

// Driver Code
int main() {
  int arr[] = {64, 34, 25, 12, 22, 11, 90};
  int n = sizeof(arr) / sizeof(arr[0]);

  cout << "Original array: ";
  printArray(arr, n);

  shellSort(arr, n);

  cout << "Sorted array: ";
  printArray(arr, n);

  return 0;
}
```

```
Original array: 64 34 25 12 22 11 90
Sorted array: 11 12 22 25 34 64 90
```

Q5) Define the process involved in the analysis of heap Sort Algorithm through derivation. Code algorithm in C++ or C# or Python

Heap Sort is a comparison-based sorting technique based on a binary heap data structure.
It sorts an array by first converting it into a Max Heap, then repeatedly extracts the maximum element and places it at the end of the array.

How Heap Sort Works (Step-by-Step)

Given: Array A of n elements

Step 1: Build Max Heap

- Convert the array into a **max heap**:
  - A complete binary tree where each parent ≥ its children.
- Largest value is at the root (A[0]).

Step 2: Heap Sort Process

- Swap root (max) with last element.
- Reduce heap size by 1.
- Call heapify() on root to restore heap property.
- Repeat until heap size = 1.

Example: [4, 10, 3, 5, 1]

Build Max Heap:

→ [10, 5, 3, 4, 1]

Sorting Steps:

Swap 10 with 1 → [1, 5, 3, 4, 10]
Heapify → [5, 4, 3, 1, 10]
Swap 5 with 1 → [1, 4, 3, 5, 10]
Heapify → [4, 1, 3, 5, 10]
... and so on
Final result: [1, 3, 4, 5, 10]

Derivation of Time Complexity

```
void heapify(int arr[], int n, int i) {
  int largest = i;        // 1
  int left = 2 * i + 1;    // 1
```

```
  int right = 2 * i + 2;    // 1

  if (left < n && arr[left] > arr[largest])
    largest = left;       // 1

  if (right < n && arr[right] > arr[largest])
    largest = right;      // 1

  if (largest != i) {
    swap(arr[i], arr[largest]);  // 1
    heapify(arr, n, largest);    // Recursive call
  }
}

void heapSort(int arr[], int n) {
  // Build Max Heap
  for (int i = n / 2 - 1; i >= 0; i--)
    heapify(arr, n, i);   // O(N)

  // Extract elements from heap
  for (int i = n - 1; i > 0; i--) {
    swap(arr[0], arr[i]);        // 1
    heapify(arr, i, 0);          // O(log N)
  }
}
```

Step 1: Building the Max Heap

- Done by calling heapify() from bottom non-leaf nodes up.
- Total cost: $\mathbf{T_{build}=O(N)}$
  (due to the fact that lower levels have smaller heights and dominate less).

Step 2: Extract Elements (Heap Sort)

- For each element, swap root with the last, then call heapify() → height of tree is log N.
- Done for N elements: $\mathbf{T_{sort}=O(NlogN)}$

Overall Time Complexity

$T(N)=O(N)+O(NlogN)=O(NlogN)$

C++ Code

```
#include <iostream>
using namespace std;

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
  int largest = i;       // Initialize largest as root
  int left = 2 * i + 1;  // Left child index
  int right = 2 * i + 2; // Right child index

  // If left child is larger than root
  if (left < n && arr[left] > arr[largest])
    largest = left;

  // If right child is larger than largest so far
  if (right < n && arr[right] > arr[largest])
    largest = right;

  // If largest is not root
```

```
    if (largest != i) {
        swap(arr[i], arr[largest]);

        // Recursively heapify the affected subtree
        heapify(arr, n, largest);
    }
}

// Main function to do heap sort
void heapSort(int arr[], int n) {
    // Step 1: Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Step 2: One by one extract elements
    for (int i = n - 1; i > 0; i--) {
        // Move current root to end
        swap(arr[0], arr[i]);

        // Call max heapify on the reduced heap
        heapify(arr, i, 0);
    }
}

// Utility function to print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver code
int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: ";
    printArray(arr, n);

    heapSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    return 0;
}
```

```
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13
```